

Отчет

Лабораторная работа №5

Азарцова Вероника Валерьевна

Содержание

1	Цель работы	5
2	Теоретическое введение	6
3	Задание	8
4	Выполнение лабораторной работы	9
4.1	Создание программы	9
4.2	Исследование Sticky-бита	16
5	Выводы	19
	Список литературы	20

Список иллюстраций

4.1	Вход от имени пользователя guest	9
4.2	Создание файла simpleid.c	10
4.3	Пункт 3-5: запуск и сравнение simpleid и id	11
4.4	Создание simpleid2.c	11
4.5	Запуск simpleid2.c	12
4.6	Команды chown и chmod	12
4.7	Проверка установки новых атрибутов и смены владельца	13
4.8	Сравнение simpleid2 и id	13
4.9	Создание readfile.c	13
4.10	Проверка чтения файла от guest	15
4.11	Проверка чтения файла с readfile	15
4.12	Проверка чтения shadow с readfile	15
4.13	Проверка tmp на атрибут Sticky	16
4.14	Создание файла с текстом, добавление прав от лица третьего польз. .	16
4.15	Чтение файла от имени guest2	16
4.16	Попытка перезаписать файл	17
4.17	Попытка дописать в файл	17
4.18	Попытка удалить файл	17
4.19	Пункт 10-12: снятие атрибута Sticky	18
4.20	Повтор предыдущий действий без Sticky	18

Список таблиц

1 Цель работы

Целью данной работы является изучение механизмов изменения идентификаторов, применения SetUID- и Sticky-битов. Получение практических навыков работы в консоли с дополнительными атрибутами. Рассмотрение работы механизма смены идентификатора процессов пользователей, а также влияние бита Sticky на запись и удаление файлов.

2 Теоретическое введение

1. Дополнительные атрибуты файлов Linux

В Linux существует три основных вида прав — право на чтение (read), запись (write) и выполнение (execute), а также три категории пользователей, к которым они могут применяться — владелец файла (user), группа владельца (group) и все остальные (others). Но, кроме прав чтения, выполнения и записи, есть еще три дополнительных атрибута. [u]

Sticky bit

Используется в основном для каталогов, чтобы защитить в них файлы. В такой каталог может писать любой пользователь. Но, из такой директории пользователь может удалить только те файлы, владельцем которых он является. Примером может служить директория /tmp, в которой запись открыта для всех пользователей, но нежелательно удаление чужих файлов.

SUID (Set User ID)

Атрибут исполняемого файла, позволяющий запустить его с правами владельца. В Linux приложение запускается с правами пользователя, запустившего указанное приложение. Это обеспечивает дополнительную безопасность т.к. процесс с правами пользователя не сможет получить доступ к важным системным файлам, которые принадлежат пользователю root.

SGID (Set Group ID)

Аналогичен suid, но относится к группе. Если установить sgid для каталога, то все файлы созданные в нем, при запуске будут принимать идентификатор группы

каталога, а не группы владельца, который создал файл в этом каталоге.

Обозначение атрибутов sticky, suid, sgid

Специальные права используются довольно редко, поэтому при выводе программы `ls -l` символ, обозначающий указанные атрибуты, закрывает символ стандартных прав доступа.

Пример: `rwsrwsrwt`

где первая `s` — это `suid`, вторая `s` — это `sgid`, а последняя `t` — это `sticky bit`

В приведенном примере не понятно, `rwt` — это `rw-` или `rwX`? Определить это просто. Если `t` маленькое, значит `x` установлен. Если `T` большое, значит `x` не установлен. То же самое правило распространяется и на `s`.

В числовом эквиваленте данные атрибуты определяются первым символом при четырехзначном обозначении (который часто опускается при назначении прав), например в правах `1777` — символ `1` обозначает `sticky bit`. Остальные атрибуты имеют следующие числовое соответствие:

1 — `sticky bit`

2 — `sgid`

4 — `suid`

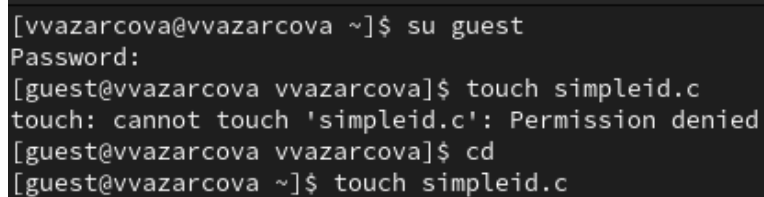
3 Задание

1. Выполнить часть 1 лабораторной работы
2. Выполнить часть 2 лабораторной работы
3. Ответить на все вопросы по процессу лабораторной работы.

4 Выполнение лабораторной работы

4.1 Создание программы

1. Осуществляется вход от имени пользователя guest (рис. 4.1).



```
[vvazarcova@vvazarcova ~]$ su guest
Password:
[guest@vvazarcova vvazarcova]$ touch simpleid.c
touch: cannot touch 'simpleid.c': Permission denied
[guest@vvazarcova vvazarcova]$ cd
[guest@vvazarcova ~]$ touch simpleid.c
```

Рисунок 4.1: Вход от имени пользователя guest

2. Создание файла simpleid.c и запись в файл кода (рис. 4.2).



```
GNU nano 5.6.1 simpleid.c Modified
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
int
main ()
{
uid_t uid = geteuid ();
gid_t gid = getegid ();
printf ("uid=%d, gid=%d\n", uid, gid);
return 0;
}

[ Read 11 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

Рисунок 4.2: Создание файла simpleid.c

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
int
main ()
{
uid_t uid = geteuid ();
gid_t gid = getegid ();
printf ("uid=%d, gid=%d\n", uid, gid);
return 0;
}
```

3. Компилирую файл, проверяю, что он скомпилировался
4. Запускаю исполняемый файл. В выводе файла выписаны номера пользователя и групп.
5. Запускаю id. От вывода при вводе id, они отличаются только тем, что

информации меньше. На снимке экрана для краткости предоставлены последние 3 действия (пункт 3-5) (рис. 4.3).

```
[guest@vvazarcova ~]$ nano simpleid.c
[guest@vvazarcova ~]$ gcc simpleid.c -o simpleid
[guest@vvazarcova ~]$ ls
dir1 simpleid simpleid.c
[guest@vvazarcova ~]$ ./simpleid
uid=1001, gid=1001
[guest@vvazarcova ~]$ id
uid=1001(guest) gid=1001(guest) groups=1001(guest) context=unconfined_u:unconfined_r:unconfined_t:s0-s0/c0-c1023
```

Рисунок 4.3: Пункт 3-5: запуск и сравнение simpleid и id

6. Создание, запись в файл и компиляция файла simpleid2.c. (рис. 4.4).

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
int
main ()
{
    uid_t real_uid = getuid ();
    uid_t e_uid = geteuid ();
    gid_t real_gid = getgid ();
    gid_t e_gid = getegid ();
    printf ("e_uid=%d, e_gid=%d\n", e_uid, e_gid);
    printf ("real_uid=%d, real_gid=%d\n", real_uid,
    real_gid);↵
    return 0;
}
```

Рисунок 4.4: Создание simpleid2.c

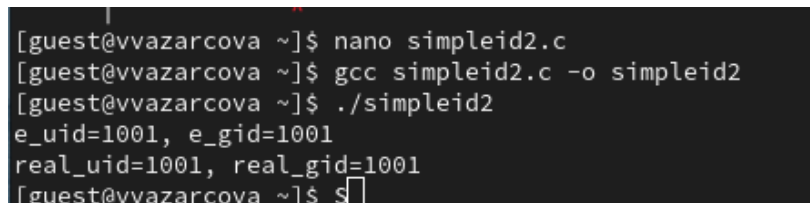
```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
int
main ()
{
    uid_t real_uid = getuid ();
```

```

uid_t e_uid = geteuid ();
gid_t real_gid = getgid ();
gid_t e_gid = getegid () ;
printf ("e_uid=%d, e_gid=%d\n", e_uid, e_gid);
printf ("real_uid=%d, real_gid=%d\n", real_uid,
real_gid);
return 0;
}

```

7. Запускаю программу (рис. 4.5).



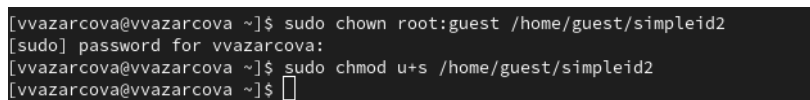
```

[guest@vvazarcova ~]$ nano simpleid2.c
[guest@vvazarcova ~]$ gcc simpleid2.c -o simpleid2
[guest@vvazarcova ~]$ ./simpleid2
e_uid=1001, e_gid=1001
real_uid=1001, real_gid=1001
[guest@vvazarcova ~]$ s

```

Рисунок 4.5: Запуск simpleid2.c

8. Использую команды chown и chmod (рис. 4.6).



```

[vvazarcova@vvazarcova ~]$ sudo chown root:guest /home/guest/simpleid2
[sudo] password for vvazarcova:
[vvazarcova@vvazarcova ~]$ sudo chmod u+s /home/guest/simpleid2
[vvazarcova@vvazarcova ~]$ 

```

Рисунок 4.6: Команды chown и chmod

9. Пояснение - с помощью chown изменяю владельца файла на суперпользователя, с помощью chmod изменяю права доступа
10. Выполняю проверку правильности установки новых атрибутов и смены владельца файла simpleid2. (рис. 4.7).

```

real_uid=1001, real_gid=1001
[guest@vvazarcova ~]$ ./simpleid2
e_uid=0, e_gid=1001
real_uid=1001, real_gid=1001
[guest@vvazarcova ~]$ 
[guest@vvazarcova ~]$ 
[vvazarcova@vvazarcova ~]$ ls -l simpleid2
ls: cannot access 'simpleid2': No such file or directory
[vvazarcova@vvazarcova ~]$ 

```

Рисунок 4.7: Проверка установки новых атрибутов и смены владельца

11. Сравнение вывода программы и команды `id`, наша команда снова вывела только ограниченное количество информации (рис. 4.8).

```

[vvazarcova@vvazarcova ~]$ /home/guest/simpleid2
e_uid=0, e_gid=1000
real_uid=1000, real_gid=1000
[vvazarcova@vvazarcova ~]$ id
uid=1000(vvazarcova) gid=1000(vvazarcova) groups=1000(vvazarcova),10(wheel) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
[vvazarcova@vvazarcova ~]$ 

```

Рисунок 4.8: Сравнение `simpleid2` и `id`

12. Проделываю то же самое относительно SetGID-бита.
13. Создание и компиляция файла `readfile.c` (рис. 4.9).

```

GNU nano 5.6.1                                readfile.c
#include <fcntl.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>
int
main (int argc, char* argv[])
{
    unsigned char buffer[16];
    size_t bytes_read;
    int i;
    int fd = open (argv[1], O_RDONLY);
    do
    {
        bytes_read = read (fd, buffer, sizeof (buffer));
        for (i = 0; i < bytes_read; ++i) printf("%c", buffer[i]);
    }
    while (bytes_read == sizeof (buffer));
    close (fd);
    return 0;
}
[ Read 21 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^N Replace   ^U Paste     ^J Justify   ^_ Go To Line
[vvazarcova@vvazarcova ~]$ ls -l / | grep tmp

```

Рисунок 4.9: Создание `readfile.c`

```

#include <fcntl.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>
int
main (int argc, char* argv[])
{
    unsigned char buffer[16];
    size_t bytes_read;
    int i;
    int fd = open (argv[1], O_RDONLY);
    do
    {
        bytes_read = read (fd, buffer, sizeof (buffer));
        for (i = 0; i < bytes_read; ++i) printf("%c", buffer[i]);
    }
    while (bytes_read == sizeof (buffer));
    close (fd);
    return 0;
}

```

14. Компилирую файл.
15. Снова от имени суперпользователя меняю владельца файла readfile. Далее меняю права доступа так, чтобы пользователь guest не смог прочесть содержимое файла.
16. Проверка чтения файл от имени пользователя guest. Прочесть файл не удастся - пункты 15, 16 (рис. 4.10).

```
[vvazarcova@vvazarcova ~]$ sudo chown root:root /home/guest/readfile.c
[vvazarcova@vvazarcova ~]$ sudo chmod 600 /home/guest/readfile.c
[vvazarcova@vvazarcova ~]$ 
[guest@vvazarcova ~]$ cat readfile.c
cat: readfile.c: No such file or directory
[guest@vvazarcova ~]$ cat readfile.c
cat: readfile.c: Permission denied
[guest@vvazarcova ~]$
```

Рисунок 4.10: Проверка чтения файла от guest

17. Попытка прочесть тот же файл с помощью программы readfile. (рис. 4.11).

```
[vvazarcova@vvazarcova ~]$ sudo chown root:guest /home/guest/readfile
[vvazarcova@vvazarcova ~]$ sudo chmod u+s /home/guest/readfile
[vvazarcova@vvazarcova ~]$ 
[guest@vvazarcova ~]$ ./readfile ./readfile.c
#include <fcntl.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>
int
main (int argc, char* argv[])
{
    unsigned char buffer[16];
    size_t bytes_read;
    int i;
    int fd = open (argv[1], O_RDONLY);
    do
    {
        bytes_read = read (fd, buffer, sizeof (buffer));
        for (i = 0; i < bytes_read; ++i) printf("%c", buffer[i]);
    }
    while (bytes_read == sizeof (buffer));
    close (fd);
    return 0;
}
[guest@vvazarcova ~]$
```

Рисунок 4.11: Проверка чтения файла с readfile

18. Попытка прочесть файл /etc/shadow с помощью программы (рис. 4.12).

```
[guest@vvazarcova ~]$ ./readfile /etc/shadow
root:$6$ZWxy04AXQZf0Vkuq$js3joK0XQBrpw5fdMjupi2dQXTtU04iugjtXDvNX8uL4QzLccfobAM
2.tjNcx8dDBrVqm5sxyoHucAYP4czz0::0:99999:7:::
bin:!:19820:0:99999:7:::
daemon:!:19820:0:99999:7:::
adm:!:19820:0:99999:7:::
lp:!:19820:0:99999:7:::
sync:!:19820:0:99999:7:::
shutdown:!:19820:0:99999:7:::
halt:!:19820:0:99999:7:::
mail:!:19820:0:99999:7:::
operator:!:19820:0:99999:7:::
games:!:19820:0:99999:7:::
ftp:!:19820:0:99999:7:::
nobody:!:19820:0:99999:7:::
systemd-coredump:!!:20501:!:!!!!:
```

Рисунок 4.12: Проверка чтения shadow с readfile

19. Пробуем прочесть эти же файлы от имени суперпользователя и чтение файлов проходит успешно.

4.2 Исследование Sticky-бита

1. Проверяем папку tmp на наличие атрибута Sticky, т.к. в выводе есть буква t, то атрибут установлен (рис. 4.13).

```
v vazarcova@v vazarcova ~]$ ls -l / | grep tmp
-rwxrwxrwt. 21 root root 4096 Apr  3 17:07 tmp
v vazarcova@v vazarcova ~]$
```

Рисунок 4.13: Проверка tmp на атрибут Sticky

2. Вхожу под именем пользователя guest2 с третьей консоли для удобства.
3. От имени пользователя guest создаю файл с текстом, добавляю права на чтение и запись для других пользователей (рис. 4.14).

```
[guest@v vazarcova ~]$ ls -l /tmp/file01.txt
-rw-r--r--. 1 guest guest 5 Apr  3 18:32 /tmp/file01.txt
[guest@v vazarcova ~]$ chmod o+rw /tmp/file01.txt
[guest@v vazarcova ~]$ ls -l /tmp/file01.txt
-rw-r--rw-. 1 guest guest 5 Apr  3 18:32 /tmp/file01.txt
[guest@v vazarcova ~]$
```

Рисунок 4.14: Создание файла с текстом, добавление прав от лица третьего польз.

4. Вхожу в систему от имени пользователя guest2, от его имени могу прочитать файл file01.txt (рис. 4.15).

```
[guest2@v vazarcova v vazarcova]$ cat /tmp/file01.txt
test
[guest2@v vazarcova v vazarcova]$
```

Рисунок 4.15: Чтение файла от имени guest2

5. Перезаписать информацию в нем не могу (рис. 4.16).

```
guest2@vvezarcova:~$ echo "test2" > /tmp/file01.txt
bash: /tmp/file01.txt: Permission denied
[guest2@vvezarcova vvezarcova]$
```

Рисунок 4.16: Попытка перезаписать файл

6. По cat очевидно видно что ничего не поменялось.
7. Также невозможно добавить в файл file01.txt новую информацию от имени пользователя guest2 (рис. 4.17).

```
guest2@vvezarcova:~$ echo "test3" > /tmp/file01.txt
bash: /tmp/file01.txt: Permission denied
[guest2@vvezarcova vvezarcova]$
```

Рисунок 4.17: Попытка дописать в файл

8. По cat очевидно снова видно что ничего не поменялось.
9. Далее пробуем удалить файл, снова получаем отказ (рис. 4.18).

```
guest2@vvezarcova:~$ rm /tmp/file01.txt
rm: cannot remove '/tmp/file01.txt': No such file or directory
[guest2@vvezarcova vvezarcova]$
```

Рисунок 4.18: Попытка удалить файл

10. От имени суперпользователя снимаем с директории атрибут Sticky
11. Покидаем режим суперпользователя командой exit.
12. Проверяем, что атрибут действительно снят - он снят. Для краткости на снимке экрана последние 3 действия - пункт 10, 11, 12 (рис. 4.19).

```
[guest2@vvazarcova vvazarcova]$ su -
Password:
[root@vvazarcova ~]# chmod -t /tmp
[root@vvazarcova ~]# exit
logout
[guest2@vvazarcova vvazarcova]$ s -l / | grep tmp
bash: s: command not found...
[guest2@vvazarcova vvazarcova]$ ls -l / | grep tmp
drwxrwxrwx. 21 root root 4096 Apr  3 18:39 tmp
[guest2@vvazarcova vvazarcova]$
```

Рисунок 4.19: Пункт 10-12: снятие атрибута Sticky

13. Далее был выполнен повтор предыдущих действий (рис. 4.20).

```
[guest2@vvazarcova vvazarcova]$ echo "test3" > /tmp/file01.txt
bash: /tmp/file01.txt: Permission denied
[guest2@vvazarcova vvazarcova]$ cat /tmp/file01.txt
test
[guest2@vvazarcova vvazarcova]$ rm /tmp/file01.txt
rm: cannot remove '/tmp/file01.txt': No such file or directory
[guest2@vvazarcova vvazarcova]$ rm /tmp/file01.txt
rm: remove write-protected regular file '/tmp/file01.txt'? y
[guest2@vvazarcova vvazarcova]$
```

Рисунок 4.20: Повтор предыдущий действий без Sticky

14. По результатам без Sticky-бита запись в файл и дозапись в файл осталась невозможной, зато удаление файла прошло успешно.
15. Возвращение директории tmp атрибута t от имени суперпользователя простыми уже знакомыми командами.

5 Выводы

Подводя выводы проведенной работе, мне удалось изучить механизмы изменения идентификаторов, применения SetUID и Sticky-биты, получить практические навыки работы в консоли с дополнительными атрибутами. Рассмотреть работы механизма смены идентификатора процессов пользователей, а также влияние бита Sticky на запись и удаление файлов

Список литературы

1. Медведовский И.Д., Семьянов П.В., Платонов В.В. Атака через Internet. — НПО «Мир и семья-95», 1997. — URL: <http://bugtraq.ru/library/books/attack1/index.html>
2. Медведовский И.Д., Семьянов П.В., Леонов Д.Г. Атака на Internet. — Издательство ДМК, 1999. — URL: <http://bugtraq.ru/library/books/attack/index.html>
3. Запечников С. В. и др. Информационная безопасность открытых систем. Том 1. — М.: Горячая линия -Телеком, 2006.